

cs5800-hw_1_coding_analysis

Mingxi Zhu

May 2026

1 Coding and Analysis

- **Analyze your fast exponent algorithm.** Is it $O(\log N)$ in the number of bits of the number? You will need to use large numbers to test it.

Using attached fast_exp.py to plot the graph "Time taken for fast exponentiation". Omit the noise, the curve shown on the bottom matches $O(\log n)$.

- **Decode the message:** Given the public keys $(N, e) = (156934814459, 5)$, the encoded message 73299208648, and the private key $d = 31386785261$. What is the message?

Using attached decode.py to calculate the message.
Decoded message = 5800

- **Break the Encryption:** You know the public keys being used by a recipient are $(N, e) = (252167, 11)$ and you have an encrypted message $M_e = 74904$. Write an algorithm to break the code by guessing the value

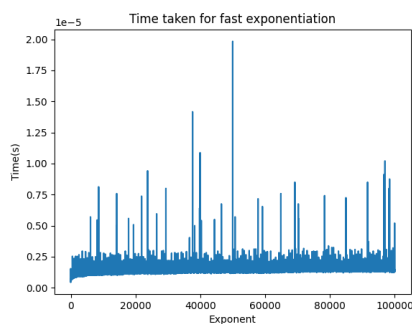


Figure 1: Time taken for fast exp for Q1

of the original message; the message has to be an integer in the range of $[1, N]$. Use the fact that $M_e = M_{\text{original}}(\hat{e}) \bmod N$.

Using the attached `break_msg.py` to break the message.

Original message = 55

- **Analyze your algorithm** What is the big-O complexity of your algorithm in terms of the number of bits of the message? What is the relationship between the message's integer value and the number of bits. What would be the run-time for your algorithm if the message size was 256 bits? Any guesses how long it would take to run?

The complexity is $O(n \log n)$

Reason: `fastExp` takes $O(\log n)$ and the linear traversal to try every possible message takes $O(n)$. Overall it takes $O(n \log n)$

2 LLM Disclosure

Use LLM to study `time.perf_counter()` in python to get precise time consumption of the fast exponentiation algorithm. Using `time.time()` is too rough to get the time in 10e-6 level.